Chapter 13

1. I was able to access the man pages for `free` with the following

```
man free
```

2. I ran with the human readable arg `-h` because I like human readable terminal output.

```
free -h
```

I got the following output:

```
ubuntu@cs5600:~$ free -h
               total        used        free      shared  buff/cache   available
Mem:           7.7Gi       406Mi       7.3Gi       4.9Mi       242Mi       7.4Gi
Swap:             0B          0B          0B
```

This matches my intuition more or less. There are some interesting things like the amount available being larger than the amount free. This implies that memory can be available but not necessarily free: most of buff/cache is reclaimable, so the kernel counts it as available even though it is not free. Only 406 Mi is actually used, which is remarkably small when we consider the fact that the average PC has between 8-16 GB of RAM. It is worth being careful here that total - free is not the same thing as used, since total = used + free + buff/cache.

3. I wrote the program memory-user.c.

4. Before running memory-user.c I made sure to take a snapshot of memory.

```
ubuntu@cs5600:~$ free -h
               total        used        free      shared  buff/cache   available
Mem:           7.7Gi       493Mi       7.1Gi       4.9Mi       463Mi       7.3Gi
Swap:             0B          0B          0B
```

I then ran to have the program allocate 100 Mi.

```
ubuntu@cs5600:~$ free -h
               total        used        free      shared  buff/cache   available
Mem:           7.7Gi       535Mi       7.0Gi       4.9Mi       463Mi       7.2Gi
Swap:             0B          0B          0B
```

I noticed that the math does not add up. Before there were 493 Mi used and my program allocated 100 Mi, so I expected to see roughly 593 Mi. Instead I measured 535 Mi, an increase of only 42 Mi. The allocation came up about 58 Mi short, not over. The reason is that malloc only reserves virtual address space; the kernel does not charge a physical page to used until the process actually touches that page, which is demand paging. My program fills the array in a loop, so this snapshot caught it partway through its first pass, and if I sampled again after the loop had touched every page I would expect the full 100 Mi to show up. The opposite effect is real too, just much smaller: the code, the stack, and

the C library all have to be resident as well, so a process always costs a little more than the bytes it explicitly allocates.

5. I read the man pages on `pmap`

```
man pmap
```

6. I chose process 901 on my VM with the name `sshd` because I recognized it from the list of processes as I skimmed through the list created by `ps auxw`.

```
root         901  0.0  0.1  14740 10476 ?        Ss   20:37   0:00 sshd: ubuntu [priv]
```

7. I see a large block of virtual addresses and their mappings as the final column.

```
ubuntu@cs5600:~/coding_2$ sudo pmap -X 901
901:   sshd: ubuntu [priv]
         Address Perm   Offset Device Inode  Size   Rss  Pss Pss_Dirty Referenced Anonymous
    57c87d675000 r--p 00000000  08:01 22939    48    48   12         0         48         0
    57c87d681000 r-xp 0000c000  08:01 22939   580   576   83         0        576         0
    57c87d712000 r--p 0009d000  08:01 22939   252   244   48         0        244         0
    57c87d751000 r--p 000dc000  08:01 22939    16    16    8         8         16        16
    57c87d755000 rw-p 000e0000  08:01 22939     4     4    4         4          4         4
    57c87d756000 rw-p 00000000  00:00     0     8     8    6         6          8         8
    57c8832d4000 rw-p 00000000  00:00     0   664   536  418       418        536       536
    797e77493000 r--p 00000000  08:01  5417    32    32    4         0         32         0
    797e7749b000 r-xp 00008000  08:01  5417   252   248   25         0        248         0
# etc ...
```

I noticed that each memory address has permissions on whether or not it can read, write, or execute. This is likely a security measure the OS needs to take. The requested vs actual shows that OS is trying to only allocate what is needed to conserve memory. There are many more memory mappings than our simple heap, stack, code model (though these are all present). I saw many more such as libraries as .so files, anonymous mappings denoted by blank entries, and mappings like vvar, vdso, and vsyscall which look like they have to do with kernel syscalls. It is clear that the address space has a lot more elements than simply code, stack, and heap addresses.

8. I added a line to my program to output the pid and ran both programs:

```
ubuntu@cs5600:~/coding_2$ ./a.out 100
PID: 2661

ubuntu@cs5600:~/coding_2$ pmap -X 2661
2661:   ./a.out 100
         Address Perm   Offset Device  Inode   Size    Rss    Pss Pss_Dirty Referenced Anony
    5a2128467000 r--p 00000000  08:01 276163      4      4      4         0          4
    5a2128468000 r-xp 00001000  08:01 276163      4      4      4         0          4
```

```
    5a2128469000 r--p 00002000  08:01 276163      4      4      4          0          4
    5a212846a000 r--p 00002000  08:01 276163      4      4      4          4          4
    5a212846b000 rw-p 00003000  08:01 276163      4      4      4          4          4
    5a21536ca000 rw-p 00000000  00:00      0    132      4      4          4          4
    7e56145ff000 rw-p 00000000  00:00      0 102404 102404 102404     102404     102404    10
    7e561aa00000 r--p 00000000  08:01   6368    160    160      5          0        160
    7e561aa28000 r-xp 00028000  08:01   6368   1572    992     31          0        992
    7e561abb1000 r--p 001b1000  08:01   6368    316    188      6          0        188
    7e561ac00000 r--p 001ff000  08:01   6368     16     16     16         16         16
    7e561ac04000 rw-p 00203000  08:01   6368      8      8      8          8          8
    7e561ac06000 rw-p 00000000  00:00      0     52     16     16         16         16
    7e561add8000 rw-p 00000000  00:00      0     12      8      8          8          8
    7e561ade1000 rw-p 00000000  00:00      0      8      4      4          4          4
    7e561ade3000 r--p 00000000  08:01   6365      4      4      0          0          4
    7e561ade4000 r-xp 00001000  08:01   6365    172    172      5          0        172
    7e561ae0f000 r--p 0002c000  08:01   6365     40     40      1          0         40
    7e561ae19000 r--p 00036000  08:01   6365      8      8      8          8          8
    7e561ae1b000 rw-p 00038000  08:01   6365      8      8      8          8          8
    7fff318f4000 rw-p 00000000  00:00      0    132     12     12         12         12
    7fff3197b000 r--p 00000000  00:00      0     16      0      0          0          0
    7fff3197f000 r-xp 00000000  00:00      0      8      4      0          0          4
ffffffffff600000 --xp 00000000  00:00      0      4      0      0          0          0
                                             ====== ====== ====== ========== ========== =====
                                             105092 104068 102556     102496     104068    10
```

I recognize the a.out mappings as the compiled binary itself, then I see that my 100 Mi allocation shows up as the anonymous 102404 KB region. I would have expected it to go in the line that says [heap], but that mapping is only size 132. The reason is that glibc's malloc only extends the heap with brk for small requests; anything at or above MMAP_THRESHOLD, which is 128 KiB by default, gets its own mmap call instead, so a 100 Mi request lands in a standalone anonymous mapping rather than growing [heap]. It also makes sense that we see libc.so.6, since my program is dynamically linked against the C library and calls printf and malloc out of it. The headers I included, such as stdio.h and stdbool.h, are compile time text and are not what produces that mapping; stdbool.h in particular is nothing but macros. The ld-linux-x86-64.so.2 mappings are the dynamic linker, which runs at process startup to map the shared libraries in and resolve relocations, so it is a runtime component rather than anything gcc did at compile time. We see again the linux specific mappings are used for this program as well.

Chapter 14

1. I wrote the program null.c where I allocated memory for an int, overwrote the pointer with NULL, then tried to dereference it. Overwriting the

pointer also throws away the only reference to that allocation, which comes back in question 3. I got a segmentation fault:

```
ubuntu@cs5600:~/coding_2$ ./null
Segmentation fault (core dumped)
```

2. I reran with gdb and got the following output:

```
Starting program: /home/ubuntu/coding_2/null
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
0x0000555555555196 in main ()
```

It would appear to me that SIGSEGV is some sort of signal sent from the operating system to kill the program because it is breaking the rules. We have the function that caused this main. The hex number is definitely an address, maybe to the instruction that crashed the program.

3. Running valgrind I got the following:

```
ubuntu@cs5600:~/coding_2$ valgrind --leak-check=yes ./null
==3034== Memcheck, a memory error detector
==3034== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==3034== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==3034== Command: ./null
==3034==
==3034== Invalid read of size 4
==3034==    at 0x4001196: main (in /home/ubuntu/coding_2/null)
==3034==  Address 0x0 is not stack'd, malloc'd or (recently) free'd
==3034==
==3034==
==3034== Process terminating with default action of signal 11 (SIGSEGV)
==3034==  Access not within mapped region at address 0x0
==3034==    at 0x4001196: main (in /home/ubuntu/coding_2/null)
==3034==  If you believe this happened as a result of a stack
==3034==  overflow in your program's main thread (unlikely but
==3034==  possible), you can try to increase the size of the
==3034==  main thread stack using the --main-stacksize= flag.
==3034==  The main thread stack size used in this run was 8388608.
==3034==
==3034== HEAP SUMMARY:
==3034==     in use at exit: 4 bytes in 1 blocks
==3034==   total heap usage: 1 allocs, 0 frees, 4 bytes allocated
==3034==
==3034== 4 bytes in 1 blocks are definitely lost in loss record 1 of 1
==3034==    at 0x484B80F: malloc (vg_replace_malloc.c:447)
==3034==    by 0x4001185: main (in /home/ubuntu/coding_2/null)
```

```
==3034==
==3034== LEAK SUMMARY:
==3034==    definitely lost: 4 bytes in 1 blocks
==3034==    indirectly lost: 0 bytes in 0 blocks
==3034==      possibly lost: 0 bytes in 0 blocks
==3034==    still reachable: 0 bytes in 0 blocks
==3034==         suppressed: 0 bytes in 0 blocks
==3034==
==3034== For lists of detected and suppressed errors, rerun with: -s
==3034== ERROR SUMMARY: 2 errors from 2 contexts (suppressed: 0 from 0)
Segmentation fault (core dumped)
```

It clearly is telling me the program performed an invalid read of size 4. The read is 4 bytes because *x dereferences an int*, so it reads an int's worth of bytes. There are two different addresses in this output and it is worth keeping them apart: 0x4001196 is the instruction address inside main that faulted, while 0x0 is the data address it tried to read. It tells me the signal as gdb did, and adds that 0x0 is not stack'd, malloc'd or free'd, meaning it is not inside any mapped region at all. We know that NULL is 0 in C, so 0x0 confirms that our program dereferenced a null pointer. This explains what happened (illegal read), where it happened (the faulting instruction in main), and which address was at fault (0x0). Valgrind also reports 4 bytes definitely lost, and that is the malloc on line 5 of null.c, whose only pointer I overwrote with NULL before I could ever free it.

4. I wrote no_free.c for this question. It just allocates memory for a number, assigns it, and dereferences it to access the value.

When I run gdb I get that the process exits normally.

```
(gdb) run
Starting program: /home/ubuntu/coding_2/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
The number is: 1
[Inferior 1 (process 3264) exited normally]
```

Valgrind however does show that there is a memory leak.

```
==3271== 4 bytes in 1 blocks are definitely lost in loss record 1 of 1
==3271==    at 0x484B80F: malloc (vg_replace_malloc.c:447)
==3271==    by 0x4001185: main (in /home/ubuntu/coding_2/a.out)
```

This matches what I would expect. Valgrind is a tool to detect memory leaks like this one, gdb is more focused on debugging and does not track allocations at all. For a program this short the leak is harmless in practice, since the OS reclaims the entire address space when the process exits. In a long running program such as a server or a daemon, though, a leak on a repeated code path grows without bound until the process gets killed, so it is more than just bad style.

5. I wrote the program named malloc_zero.c. When I ran this program

nothing bad seemed to happen, it just printed the "done" message I put at the end. When I run valgrind we see that I still had an invalid write and the 400 bytes are being leaked. The invalid write is an off by one: the array holds 100 ints, so the valid indices are 0 through 99, and I wrote to data[100]. There is no mention of a SIGSEGV, because that write still lands inside a page the heap already has mapped, where it hits allocator slack or metadata, so the hardware never faults even though the write is out of bounds. The two bugs are an invalid write (out of bounds) and the memory leak. This goes to show that a program with serious memory issues can look perfectly benign after compiling and running it.

6. I wrote the program named use_after_free.c. When running and compiling there were no errors. I did notice however that when I ran it different ways the value that was printed out was not always the same. Nothing valid lives at x[1] once the block has been freed, since the allocator is then free to reuse those bytes for its own bookkeeping, so what I read back is just whatever happens to be sitting there at the time. Valgrind did pick up on this:

```
==9588== Command: ./a.out
==9588==
==9588== Invalid read of size 4
==9588==    at 0x40011BE: main (in /home/ubuntu/coding_2/a.out)
==9588==  Address 0x4a7d044 is 4 bytes inside a block of size 400 free'd
==9588==    at 0x484EB2C: free (vg_replace_malloc.c:990)
==9588==    by 0x40011B5: main (in /home/ubuntu/coding_2/a.out)
==9588==  Block was alloc'd at
==9588==    at 0x484B80F: malloc (vg_replace_malloc.c:447)
==9588==    by 0x40011A5: main (in /home/ubuntu/coding_2/a.out)
==9588==
```

It shows that there was a free and then it was tried to be used. This still falls into the classification of an invalid read.

7. I wrote the program funny_free.c. It compiled but with warnings.

```
ubuntu@cs5600:~/coding_2$ gcc funny_free.c
funny_free.c: In function ‘main’:
funny_free.c:6:3: warning: ‘free’ called on pointer ‘x’ with nonzero offset 196 [-Wfree-nonh
    6 |   free(x + 49);
      |   ^~~~~~~~~~~~
funny_free.c:5:13: note: returned from ‘malloc’
    5 |   int * x = malloc(sizeof(int) * 100);
      |             ^~~~~~~~~~~~~~~~~~~~~~~~~
```

When I ran the compiled program I got the following.

```
ubuntu@cs5600:~/coding_2$ ./a.out
free(): invalid pointer
```

```
Aborted (core dumped)
```

You definitely do need tools because despite the compiler warnings the program did compile. gcc could only warn here because the offset 49 is a compile time constant that it can fold; if I had written free(x + i) with i computed at runtime, the warning would disappear entirely while valgrind would still catch it. Valgrind reports this as an invalid free, and an all around bad unpredictable thing to do.

```
ubuntu@cs5600:~/coding_2$ valgrind --leak-check=yes ./a.out
==9914== Memcheck, a memory error detector
==9914== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==9914== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==9914== Command: ./a.out
==9914==
==9914== Invalid free() / delete / delete[] / realloc()
==9914==    at 0x484EB2C: free (vg_replace_malloc.c:990)
==9914==    by 0x40011BB: main (in /home/ubuntu/coding_2/a.out)
==9914==  Address 0x4a7d104 is 196 bytes inside a block of size 400 alloc'd
==9914==    at 0x484B80F: malloc (vg_replace_malloc.c:447)
==9914==    by 0x40011A5: main (in /home/ubuntu/coding_2/a.out)
==9914==
==9914== Conditional jump or move depends on uninitialised value(s)
==9914==    at 0x48D10CB: __printf_buffer (vfprintf-process-arg.c:58)
==9914==    by 0x48D273A: __vfprintf_internal (vfprintf-internal.c:1544)
==9914==    by 0x48C71B2: printf (printf.c:33)
==9914==    by 0x40011DB: main (in /home/ubuntu/coding_2/a.out)
==9914==
==9914== Use of uninitialised value of size 8
==9914==    at 0x48C60BB: _itoa_word (_itoa.c:183)
==9914==    by 0x48D0C9B: __printf_buffer (vfprintf-process-arg.c:155)
==9914==    by 0x48D273A: __vfprintf_internal (vfprintf-internal.c:1544)
==9914==    by 0x48C71B2: printf (printf.c:33)
==9914==    by 0x40011DB: main (in /home/ubuntu/coding_2/a.out)
==9914==
==9914== Conditional jump or move depends on uninitialised value(s)
==9914==    at 0x48C60CC: _itoa_word (_itoa.c:183)
==9914==    by 0x48D0C9B: __printf_buffer (vfprintf-process-arg.c:155)
==9914==    by 0x48D273A: __vfprintf_internal (vfprintf-internal.c:1544)
==9914==    by 0x48C71B2: printf (printf.c:33)
==9914==    by 0x40011DB: main (in /home/ubuntu/coding_2/a.out)
==9914==
==9914== Conditional jump or move depends on uninitialised value(s)
==9914==    at 0x48D0D85: __printf_buffer (vfprintf-process-arg.c:186)
==9914==    by 0x48D273A: __vfprintf_internal (vfprintf-internal.c:1544)
==9914==    by 0x48C71B2: printf (printf.c:33)
==9914==    by 0x40011DB: main (in /home/ubuntu/coding_2/a.out)
==9914==
```

```
The number is: 0
==9914==
==9914== HEAP SUMMARY:
==9914==     in use at exit: 400 bytes in 1 blocks
==9914==   total heap usage: 2 allocs, 2 frees, 1,424 bytes allocated
==9914==
==9914== 400 bytes in 1 blocks are definitely lost in loss record 1 of 1
==9914==    at 0x484B80F: malloc (vg_replace_malloc.c:447)
==9914==    by 0x40011A5: main (in /home/ubuntu/coding_2/a.out)
==9914==
==9914== LEAK SUMMARY:
==9914==    definitely lost: 400 bytes in 1 blocks
==9914==    indirectly lost: 0 bytes in 0 blocks
==9914==      possibly lost: 0 bytes in 0 blocks
==9914==    still reachable: 0 bytes in 0 blocks
==9914==         suppressed: 0 bytes in 0 blocks
==9914==
==9914== Use --track-origins=yes to see where uninitialised values come from
==9914== For lists of detected and suppressed errors, rerun with: -s
==9914== ERROR SUMMARY: 6 errors from 6 contexts (suppressed: 0 from 0)
```

Note that valgrind reports 6 errors, not just the invalid free. The other four all come from printing x[1], which I never initialized, so printf ends up branching on uninitialised values. That is a second, separate bug in this program that I would not have noticed without the tool.

8. I wrote vector.c for this question. I found it wasn't that difficult to write a program that functioned reasonably well. I ran with valgrind and got the following leak summary:

```
==10458== LEAK SUMMARY:
==10458==    definitely lost: 0 bytes in 0 blocks
==10458==    indirectly lost: 0 bytes in 0 blocks
==10458==      possibly lost: 0 bytes in 0 blocks
==10458==    still reachable: 2,076 bytes in 3 blocks
==10458==         suppressed: 0 bytes in 0 blocks
```

Nothing is definitely lost, but the 2,076 bytes still reachable are not a clean bill of health. Still reachable only means the pointer was still live at exit, not that I released the memory. I never call free(vec), so that is the vector itself sitting there unfreed.

On performance, my implementation reallocs by exactly one element on every insert, so each insert can copy the whole array to keep it contiguous and appending n elements costs O(n^2) overall. That is the worst case I expected to see, but it is a property of my growth strategy rather than of vectors. The standard fix is to grow geometrically, usually by doubling the capacity, which makes append amortized O(1) because the expensive copies get exponentially rarer as

the array grows. With doubling I would expect the vector to beat a linked list rather than lose to it: a linked list pays a separate malloc and 8 to 16 bytes of pointer overhead for every single element, and walking it chases pointers scattered across the heap, which costs roughly a cache miss per node, whereas the vector's elements are contiguous and prefetch well. So the real comparison here is not vector versus linked list, it is my naive growth strategy versus a good one.

9. I spent some time browsing through the man pages for valgrind and gdb.